Linux System Administration (E1ITA307)

Linux Infrastructure Deployment Project Report

On

COLLEGE COMPUTER LABORATORY SERVER MANAGEMENT

(LabServer)

Submitted by

SHASHANK PATEL (2411021061117)
PANKAJ PANDURANG TIKE (2411021061162)
ANMOL PODDAR (2411021061144)
SHIKHAR SRIVASTAVA (2411021061140)

In partial fulfillment of the award of the degree of

BACHELOR OF TECHNOLOGY

In

Information Technology

Under the Supervision of

Mrs. KUSUMA J.

ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY

BENGALURU

Nov 2026



DEPARTMENT OF INFORMATION TECHNOLOGY

ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the Project work entitled **"College Computer Laboratory Server Management"** submitted by Shashank Patel [2411021061117], Pankaj Pandurang Tike [2411021061162], Anmol Poddar [2411021061144] and Shikhar Srivastava [2411021061140] in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology of Alliance University, is a bonafide work during the academic year 2026-2027.

Mrs. Kusuma J. Course Handling Faculty	Prof. Ananthanagu U Course Coordinator	Dr. Shekhar R Professor & HoD

Table of Contents

1. Introduction....4
1.1 Aim....4
1.2 Background....4
1.3 Problem Statement....4
1.4 Objectives....4
1.5 Team and Responsibilities....5
1.6 Scope of the Project....5
1.7 Organisation of the Report....5
2. Methodology....6
2.1 Tools and Technologies Used....6
2.2 Approach / Development Process....6
3. Linux System Administration – Design and Implementation....8
3.1 System Architecture....8
3.2 Linux Environment Setup....9
3.3 User and Group Management....11
3.4 File and Directory Management....12
3.5 Package and Process Management....13
3.6 Storage and File System Configuration....14
3.7 Backup and Automation....15
3.8 Security and Access Control....16
4. Results and Testing....18
4.1 Implementation Results....18
4.2 Testing and Validation....19
4.3 Observations....19
5. Conclusion and Future Scope....21
5.1 Conclusion....21
5.2 Future Scope....21
References....22
Appendices....23
Appendix A: Dashboard Pages and Data....23
Appendix B: Troubleshooting....24
Appendix C: Scripts....24

1. Introduction

1.1 Aim

The main aim of this project is to design, develop and implement a College Computer Laboratory Server Management System (LabServer) on Linux. The system automatically creates and organises student accounts, provides secure shared folders for every department, checks that the required lab software is installed, cleans old temporary files, takes backups and proves that they can be restored, monitors CPU, memory and disk usage, and presents everything as a terminal health report and a web dashboard.

1.2 Background

A college computer laboratory is a shared Linux environment used by many students from several departments. Every semester the administrator has to create accounts, give each department a private working area, keep the lab software available, remove files that fill the disk, and keep backups. Done by hand, these tasks are slow and easy to get wrong; a missed permission lets one department read another's work, and a backup that was never tested may fail on the day it is needed.

Our system moves this work into one Bash script (`server/collect.sh`, with an equivalent Python version `server/collect.py`) that uses standard Linux tools such as `useradd`, `chmod`, `find` and `tar`. The script writes a structured `data.json` file, and a React website shows the same numbers on a dashboard. The website runs on macOS, Linux and Windows; the script itself needs Linux because it creates real users.

1.3 Problem Statement

College computer laboratories need many student accounts, separate shared storage for each department, a standard set of software, regular cleanup, dependable backups and a way to watch server health. Managing these by hand is time-consuming, error-prone and hard to audit. The administrator needs a repeatable, automated way to set up and maintain the lab server, and one place to see its current state.

1.4 Objectives

- To create 100 student accounts (25 in each of four departments) automatically from a CSV list, with a forced password change at first login.
- To give each department a shared folder that only its own members can open, using group permissions and the setgid bit (mode 2770).
- To verify that the standard lab software (gcc, Python 3, Java, Git, Node.js, make, vim, curl) is installed and report its version.
- To clean temporary files older than seven days automatically and report how many files and how much space were recovered.
- To back up the department folders to a compressed archive and prove, by a restore test, that the backup works.
- To monitor CPU, memory, disk usage, uptime and logged-in users, and print a health report.
- To schedule the whole routine with cron and show the results on a web dashboard.

1.5 Team and Responsibilities

Table 1.1: Team members and roles

Member	Register No.	Role	GitHub
Shashank Patel	2411021061117	Linux System Administrator	allenasat044-prog
Pankaj Pandurang Tike	2411021061162	Storage & Backup Administrator	pankajt25
Anmol Poddar	2411021061144	Automation & Security Engineer	ANMOL16-IT
Shikhar Srivastava	2411021061140	Testing, Documentation & Presentation Lead	FFloverSHIKHAR2006

1.6 Scope of the Project

The Linux script does the real administration work: it creates accounts, groups, folders, cleanup, backups and the restore test. The website only displays the data saved by the script; it has no login and cannot change the server. The project covers one lab server. It does not include central authentication (LDAP), disk quotas, off-site backup or e-mail alerts; these are listed as future scope in Chapter 5. Section 3.6.4 adds a recommended LVM layout as a reference procedure.

1.7 Organisation of the Report

Table 1.2: Report organisation

Chapter	Content
1. Introduction	Aim, background, problem statement, objectives and team
2. Methodology	Tools, technologies and the development process
3. Design and Implementation	Architecture, environment, users, folders, packages, storage, backup, automation and security
4. Results and Testing	Output of the real run, validation table and scope of testing
5. Conclusion and Future Scope	Summary of the work and possible improvements
References and Appendices	Sources, dashboard pages, troubleshooting and the full script

2. Methodology

2.1 Tools and Technologies Used

2.1.1 Software Requirements

Table 2.1: Software used

Component	Used for
Ubuntu 24.04.4 LTS (virtual machine)	Lab server used for development and testing of the script
Bash (`collect.sh`)	Main collector: users, folders, cleanup, backup, restore test, monitoring
Python 3.12 (`collect.py`)	Equivalent collector written in Python
Node.js 18 or newer	Builds and runs the website (v22.22.2 on the test VM)
React 18, TypeScript, Vite 5, Tailwind CSS 4	Web dashboard (Recharts, Framer Motion, GSAP, Radix UI for charts, animation and components)
Linux utilities	`useradd`, `groupadd`, `chpasswd`, `chage`, `chmod`, `chown`, `find`, `tar`, `sha256sum`, `stat`, `df`, `who`, `ps`, `uptime`
cron	Runs the routine every night at 02:00
Git and GitHub	Version control and project hosting

2.1.2 Hardware Requirements

Any laptop, desktop or virtual machine that runs Ubuntu or another modern Linux distribution is enough. The test VM had one CPU core, 3.9 GB of memory and a 252 GB disk. The script is light: it creates 100 home directories, a few small files and one small archive.

2.1.3 Other Tools and Resources

- `students.csv`: the list of 100 usernames and their departments, created by the script.
- `server/data.json` and `src/data.json`: the saved output that the website reads.
- `package.json`: npm scripts for running, building and refreshing the data.
- Terminal and `README.md`: used to run the system and check the results.

2.2 Approach / Development Process

The project was developed in eight stages.

Step 1: Problem identification: manual account creation, mixed-up folder access, full disks, untested backups and no overview of server health.

Step 2: Requirement analysis: 100 users, four departments, private shared folders, software check, cleanup, backup with restore test, monitoring and a dashboard.

Step 3: Environment setup: an Ubuntu 24.04 VM and one project folder holding the server script, the website and the data.

Step 4: System configuration: department groups, folder base path `/srv/lab`, default password policy and the cron schedule.

Step 5: Implementation: `collect.sh` with six modules, and the React website with six pages.

Step 6: Automation: nightly cron job (`npm run refresh`) that runs the script and copies the data to the website.

Step 7: Testing: checks of accounts, folders, cleanup, backup and restore on the VM, and a type-check and build of the website.

Step 8: Documentation: diagrams, configuration, commands and results collected in this report.

3. Linux System Administration – Design and Implementation

3.1 System Architecture

The system has two parts. The administration script runs on the Linux server, uses the operating system as its only data source and writes plain files. The website reads the saved `data.json` and shows it in the browser. Figure 3.1 shows the components and how they connect.

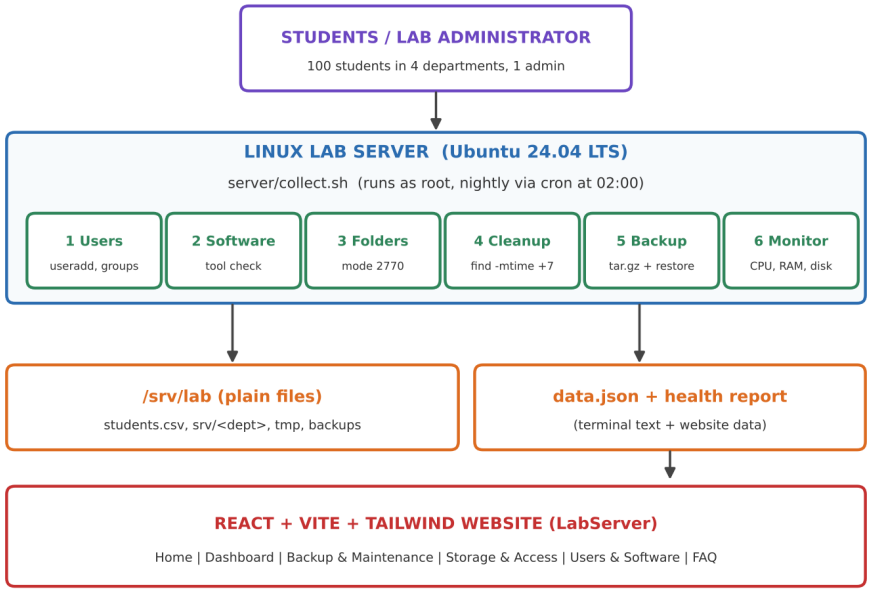


Figure 3.1: System architecture of the College Computer Laboratory Server Management System

Components and their roles

Table 3.1: Components and roles

Component	Role
1. Client / User	Students log in to their accounts and work in their department folder. The administrator runs the script and opens the dashboard.
2. Linux Server	Ubuntu 24.04 VM that runs `collect.sh` as root and provides `/proc`, `df`, `useradd`, `tar` and cron.
3. Storage	`/srv/lab` holds `students.csv`, the department folders, the temp folder and the backups.
4. Network	The script needs no network. The website runs on the Vite server (`localhost:5173`); the landing image and the storage video load from the internet.
5. Applications / Services	`collect.sh` (and `collect.py`) for administration, and the React dashboard for display.
6. Backup System	Compressed `tar.gz` archive of the department folders with an automatic restore test.
7. Monitoring and Automation	CPU, memory and disk readings; nightly cron job; terminal report and `data.json`.

The script runs its six modules in a fixed order, shown in Figure 3.2. Each module is safe to run again: existing users are skipped, folders are re-created with the same mode, and every backup gets a new time-stamped name.

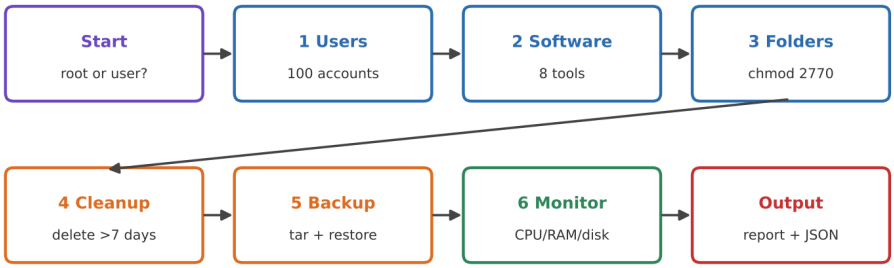


Figure 3.2: Order of the six modules in collect.sh

3.2 Linux Environment Setup

3.2.1 Platform and Requirements

The script runs on Linux with Bash and the standard GNU tools. It was tested on Ubuntu 24.04.4 LTS. It must run as root to create users and groups. When started by a normal user it still runs, but it uses `$HOME/lab-server` as its base folder and does not create accounts. Node.js 18 or newer is needed only for the website.

3.2.2 Installation Process

The project is unpacked into a folder in the home directory. The website dependencies are installed once with npm, and the script is made executable.

```
</> Bash
sudo apt update
sudo apt install -y nodejs npm unzip
cd ~/Downloads
unzip -o labserver.zip -d ~
cd ~/labserver
npm install
chmod +x server/collect.sh
sudo ./server/collect.sh
npm run dev
```

`sudo ./server/collect.sh` performs the real administration work and prints the health report. `npm run dev` starts the website; it is opened at `http://localhost:5173`.

3.2.3 Directory Structure

```
</> Project tree
labserver/
├── server/
│   ├── collect.sh        # main collector (Bash)
│   ├── collect.py        # equivalent collector (Python)
│   └── data.json         # output of the last run
├── src/
│   ├── pages/            # Landing, Dashboard, Backups, Storage, Users, Faq
│   ├── components/ui/    # cards, charts, accordion, hero
│   ├── lib/lab.ts        # reads data.json, GitHub link
│   └── data.json         # copy of the data shown on the website
```

```
├── package.json          # npm scripts (dev, build, refresh ...)
├── vite.config.ts
└── README.md
```

```
</> Server base folder (/srv/lab)
/srv/lab/
├── students.csv          # username,department for 100 students
├── srv/
│   ├── cse/    members.txt
│   ├── ece/    members.txt
│   ├── mech/   members.txt
│   └── civil/  members.txt
├── tmp/                  # lab temp files, cleaned after 7 days
└── backups/              # backup-YYYYMMDD-HHMMSS.tar.gz
```

Figure 3.3: Project and server directory structure

3.2.4 Network Configuration

Collecting the data needs no network. The website is served on port 5173 by the Vite development server, or can be built with `npm run build` and previewed with `npm run preview`. Administrators reach the server over SSH, and the saved data can be copied to another computer with `scp` (see Appendix B).

3.2.5 User Accounts

The administrator runs the script as root (with `sudo`). The 100 student accounts are ordinary users with a home directory and the `/bin/bash` shell, each in the group of their department. Account creation is described in section 3.3.

3.2.6 Required Packages

Node.js and npm are needed for the website, and `unzip` to unpack the project. The script uses only tools that are part of a standard Ubuntu installation. The software check (section 3.5.1) also looks for gcc, Python 3, Java, Git, Node.js, make, vim and curl. In the test run `vim` was missing; it can be installed with:

```
</> Bash
sudo apt install -y vim
```

3.2.7 Scheduling with cron

The nightly routine is scheduled with cron. The job runs `npm run refresh`, which executes `collect.sh` and copies the new `data.json` into the website.

```
</> Bash
sudo crontab -e
```

```
</> crontab (root)
# run every night at 02:00
0 2 * * *  cd /home/YOUR_USER/labserver && npm run refresh
```

`YOUR_USER` is replaced by the login name of the account that owns the project folder.

3.3 User and Group Management

3.3.1 User and Group Creation

The script first writes `students.csv` with 100 rows: `student001` to `student100`. Departments are assigned in turn (CSE, ECE, MECH, CIVIL), so each has 25 students. Then it creates one group per department and one user per row.

```
</> Bash (from collect.sh)
DEPTS=(cse ece mech civil)
for d in "${DEPTS[@]}"; do getent group "lab-$d" >/dev/null || groupadd "lab-$d"; done
for r in "${ROWS[@]}"; do
  u=${r%,*}; d=${r#*,}
  if ! id "$u" >/dev/null 2>&1; then
    useradd -m -s /bin/bash -g "lab-$d" "$u" && echo "$u:Lab@2026" | chpasswd && chage -d
0 "$u"
  fi
done
```

`getent group` makes the group creation safe to repeat, and `id` skips users that already exist. `useradd -m` creates the home directory, `chpasswd` sets the initial password and `chage -d 0` expires it so that every student must choose a new password at the first login. The result is checked with:

```
</> Bash
getent passwd | grep -c '^student'
id student001
getent group lab-cse
chage -l student001
```

3.3.2 File Ownership and Permissions

Every department folder under `/srv/lab/srv/` is given to the department group and set to mode `2770`.

```
</> Bash (from collect.sh)
chown ":lab-$d" "$p"
chmod 2770 "$p"
stat -c %a "$p"
```

Table 3.2: Meaning of mode 2770

Part	Meaning
2 (setgid)	New files and folders inside inherit the department group, so the whole department can use them.
7 (owner)	Read, write and enter the folder.
7 (group)	Read, write and enter the folder: all members of the department group.
0 (others)	No access for students of other departments.

3.3.3 Access Control

Members of `lab-cse` can open only the CSE folder, members of `lab-ece` only the ECE folder, and so on. A student from another department receives "Permission denied". The script also writes a `members.txt` file in each folder that lists the 25 usernames of that department (Figure 3.4).

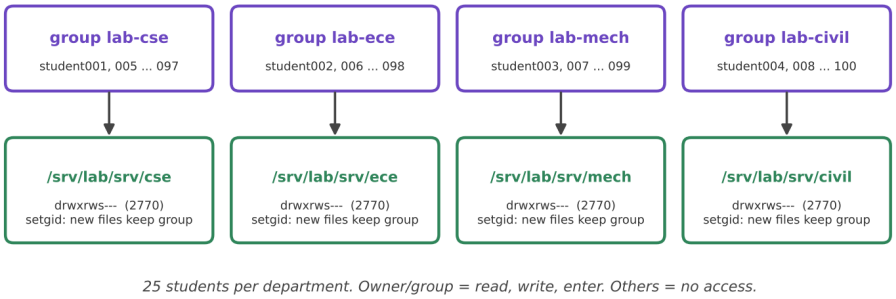


Figure 3.4: Department groups and their shared folders (mode 2770)

3.3.4 Password Management

All student accounts start with the same default password, `Lab@2026`, which is expired immediately with `chage -d 0`. At the first login the system asks the student to enter a new password. Password ageing for an account can be viewed with `chage -l`. The default password is a known limit of the project and is discussed in section 3.8.2.

3.4 File and Directory Management

3.4.1 Directory Structure

Table 3.3: Server folders

Folder	Purpose
`/srv/lab/`	Base folder of the lab server (`$HOME/lab-server` when not run as root)
`/srv/lab/srv/<dept>/`	Shared working folder of one department, mode 2770, with `members.txt`
`/srv/lab/tmp/`	Temporary files; files older than 7 days are deleted
`/srv/lab/backups/`	Time-stamped `tar.gz` backups
`labserver/server/`	`collect.sh`, `collect.py` and the last `data.json`
`labserver/src/`	Website source code and the copy of `data.json` it displays

3.4.2 Files Written by the Script

Table 3.4: Data files

File	Content	Kept for
`students.csv`	Header `username,department` and 100 rows	Rewritten each run
`srv/<dept>/members.txt`	The 25 usernames of the department	Rewritten each run
`tmp/lab_<time>_<n>.tmp`	Test temp files (20 KB each)	Deleted after 7 days
`backups/backup-<time>.tar.gz`	Archive of `/srv/lab/srv`	Kept; last 6 listed
`server/data.json`	All results of the run as JSON	Overwritten
`src/data.json`	Copy shown on the website	Overwritten

3.4.3 Searching and Checking Files

Because everything is stored as plain files and standard tools, the administrator can check the system from the terminal.

```
</> Bash
ls -ld /srv/lab/srv/*
head -3 /srv/lab/students.csv
head -3 /srv/lab/srv/cse/members.txt
find /srv/lab/tmp -type f -mtime +7
ls -lh /srv/lab/backups
python3 -c "import json; print(json.load(open('server/data.json'))['report'])"
```

The last command prints the saved health report from `data.json` without running the script again.

3.5 Package and Process Management

3.5.1 Package Management

The script checks eight tools with `command -v` and records the first line of `<tool> --version`. A missing tool is stored as `null` and shown as "missing" on the Users & Software page.

Table 3.5: Software checked and npm scripts

Item	Purpose
gcc, python3, java, git, node, make, vim, curl	Standard lab software that must be present
`npm run dev`	Starts the website (Vite) on `http://localhost:5173`
`npm run build`	Builds the production website into `dist/`
`npm run preview`	Previews the production build
`npm run typecheck`	TypeScript check (`tsc --noEmit`)
`npm run refresh`	Runs `server/collect.sh` and copies `data.json` to `src/data.json`

3.5.2 Process Management

The monitoring module lists the users that currently own processes (`ps -eo user= | sort -u`) and the users logged in (`who`). The administrator can inspect processes and load with standard commands.

```
</> Bash
ps -eo user= | sort -u
who
uptime
ps aux --sort=-%cpu | head -5
```

3.5.3 Service Management and Logs

The nightly job is a cron entry. The administrator can confirm that cron is running and see its executions in the system log.

```
</> Bash
systemctl status cron --no-pager
grep CRON /var/log/syslog | tail -5
sudo crontab -l
```

3.6 Storage and File System Configuration

3.6.1 How the Script Measures the Server

Table 3.6: Monitoring sources

Metric	Source and method
CPU %	`/proc/stat` is read twice, one second apart. Usage = 100 × (1 − idle time / total time).
Memory	`/proc/meminfo`: used = MemTotal − MemAvailable.
Disk (/)	`df -B1 /`: used and total bytes of the root file system.
Load average	`/proc/loadavg`: 1, 5 and 15 minute values.
Uptime, host, OS	`uptime -p`, `hostname`, `/etc/os-release`.
Logged-in users	`who` and the owners of running processes.

3.6.2 Temporary File Cleanup

To demonstrate the cleanup, the script creates 30 test files of 20 KB each in `/srv/lab/tmp`: 20 files are dated 10 days ago and 10 files one day ago. It then deletes every file older than seven days and reports the result.

```
</> Bash (from collect.sh)
head -c 20480 /dev/urandom > "$f"
touch -d "10 days ago" "$f"
find "$tmp" -type f -mtime +7 -delete
```

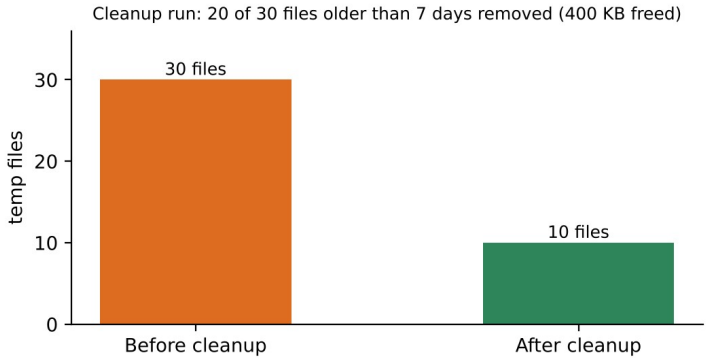


Figure 3.5: Result of the cleanup run (real sample)

3.6.3 Resource Usage

CPU, memory and disk are shown as percentages. Figure 3.6 plots the values measured on the test VM. A reading above 80 % is a good point for the administrator to look at the server more closely; the script itself only reports the numbers.

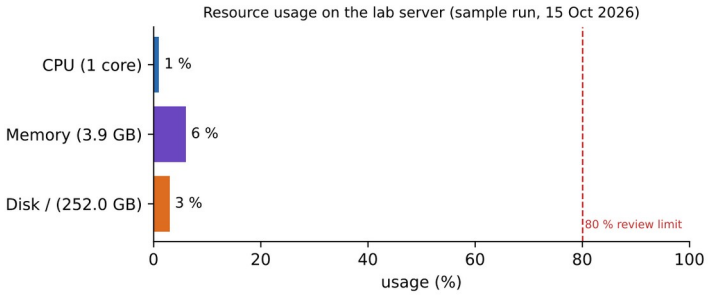


Figure 3.6: CPU, memory and disk usage of the lab server (real sample)

3.6.4 Recommended Storage Layout (LVM, reference)

The script does not create volumes; it works on whatever is mounted at `/srv/lab`. For a production lab we recommend placing `/srv/lab` on a logical volume so that it can grow when more students are added. **Note:** these commands are a reference procedure, are destructive on the disk they are pointed at, and were not run in the project tests.

```
</> Bash
sudo pvcreate /dev/sdb
sudo vgcreate vg_lab /dev/sdb
sudo lvcreate -L 50G -n lv_lab vg_lab
sudo mkfs.ext4 /dev/vg_lab/lv_lab
sudo mkdir -p /srv/lab
sudo mount /dev/vg_lab/lv_lab /srv/lab
sudo lvextend -r -L +10G /dev/vg_lab/lv_lab
```

3.7 Backup and Automation

3.7.1 Backup Strategy

Each run archives the department folders (`/srv/lab/srv`) into a compressed file named `backup-YYYYMMDD-HHMMSS.tar.gz` in `/srv/lab/backups`. The script records the size, the first 16 characters of the SHA-256 checksum and the number of files archived, and keeps the six newest archives in the backup history.

3.7.2 Backup and Restore Test

```
</> Bash (from collect.sh)
ts=$(date +%Y%m%d-%H%M%S); bf="$bd/backup-$ts.tar.gz"
tar -czf "$bf" -C "$B" srv
SRC=$(find "$B/srv" -type f | wc -l)
td=$(mktemp -d)
tar -xzf "$bf" -C "$td"
GOT=$(find "$td/srv" -type f | wc -l)
rm -rf "$td"
SHA=$(sha256sum "$bf" | cut -c1-16)
if [ "$SRC" -eq "$GOT" ]; then RESULT=PASSED; else RESULT=FAILED; fi
```



Figure 3.7: Backup creation and restore test

After creating the archive, the script extracts it into a temporary folder, counts the files and compares the count with the original. Equal counts mean the restore test is PASSED; otherwise it is FAILED. The temporary folder is removed afterwards.

3.7.3 Recovery Procedure

To recover, the administrator lists the backups, restores the wanted folder into a safe location and compares it with the original before copying files back.

```
</> Bash
ls -lh /srv/lab/backups
mkdir -p /tmp/restore
sudo tar -xzf /srv/lab/backups/backup-20261015-031647.tar.gz -C /tmp/restore
diff -r /srv/lab/srv /tmp/restore/srv && echo "no differences"
```

3.7.4 Built-in Automation

Table 3.7: Automation shipped with the project

Mechanism	What it does
cron (`0 2 * * *`)	Runs `npm run refresh` every night at 02:00
`npm run refresh`	Runs `collect.sh`, then copies `data.json` into the website
Repeatable runs	Existing users and groups are skipped; folders are re-created with the same mode
Automatic cleanup	Files older than 7 days are removed in every run
Automatic restore test	Every backup is extracted and its file count is compared
Terminal report	The same text is printed and stored in `data.json`

3.8 Security and Access Control

Table 3.8: Security controls

Control	Detail
Group-based access	Each department folder is mode 2770 and owned by its own group; other students have no access
Forced password change	`chage -d 0` expires the initial password, so students must set their own at first login
Separate roles	Only root creates accounts; run as a normal user, the script creates none and uses `$HOME/lab-server`
Safe scripting	`set -u`, quoted variables and a `j()` function that escapes text written into JSON

Control	Detail
Read-only website	The dashboard only displays saved data; it has no function that changes the server
Tested backups	Every backup is restored to a temporary folder and verified

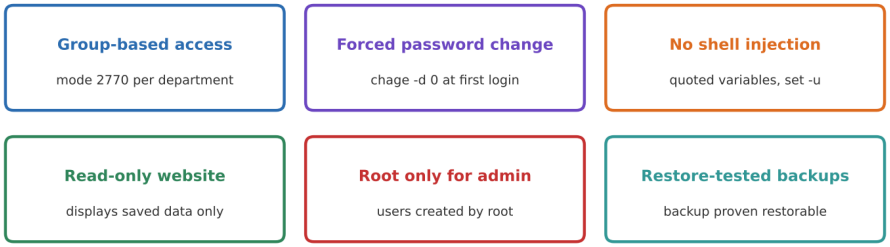


Figure 3.8: Security measures

3.8.1 Firewall and SSH

Administrators reach the server over SSH. The dashboard is a static website and should be exposed only to a trusted network. On Ubuntu, `ufw` can restrict access.

```
</> Bash
sudo ufw status verbose
sudo ufw allow OpenSSH
systemctl status ssh --no-pager
```

3.8.2 Known Limits

- All 100 accounts share the default password `Lab@2026` until each student changes it; the password is written in the script and should be moved to a protected setting before real use.
- The website has no login. `data.json` contains the host name and usernames, so the site should not be published openly.
- Backups are stored on the same server disk; a copy on another machine (for example with `scp` or `rsync`) is recommended.
- The script does not delete old backups or set disk quotas for students.

4. Results and Testing

4.1 Implementation Results

The repository contains the output of a real run of `collect.sh` as root on the Ubuntu 24.04.4 LTS test VM (host `vm`) on 15 October 2026. Figure 4.1 shows the health report exactly as it was printed.

```
</> $ sudo ./collect.sh
LAB SERVER HEALTH REPORT
Generated : 15 Oct 2026, 03:16:47
Host      : vm (Ubuntu 24.04.4 LTS)
Uptime    : up 0 minutes

CPU       : 1% of 1 cores, load [0.0, 0.0, 0.0]
Memory    : 0.2 GB / 3.9 GB (6%)
Disk (/)  : 8.5 GB / 252.0 GB (3%)

Accounts  : 100 student accounts present, 100 in students.csv
Software  : 7/8 lab tools installed
Folders   : cse(2770), ece(2770), mech(2770), civil(2770)
Cleanup   : removed 20 of 30 temp files, freed 400 KB
Backup    : backup-20261015-031647.tar.gz (0 KB), restore test PASSED (4/4
files)
```

Figure 4.1: Lab server health report (sample run)

Table 4.1: Results of the run

Item	Result
Student accounts	100 present, 100 rows in `students.csv`
Department folders	cse, ece, mech, civil: all mode 2770, groups lab-cse, lab-ece, lab-mech, lab-civil
Lab software	7 of 8 tools installed (gcc 13.3.0, Python 3.12.3, OpenJDK 21.0.10, Git 2.43.0, Node.js v22.22.2, make 4.3, curl 8.5.0); vim missing
Cleanup	20 of 30 temp files removed, 400 KB (409,600 bytes) freed
Backup	`backup-20261015-031647.tar.gz`, 707 bytes, checksum prefix `e6b27f859f49752e`
Restore test	PASSED: 4 of 4 files restored
CPU / Memory / Disk	1 % of 1 core / 0.2 GB of 3.9 GB (6 %) / 8.5 GB of 252.0 GB (3 %)

The backup is only 707 bytes because the department folders held just the four `members.txt` files at the time of the run. The size will grow when students save their work.

4.2 Testing and Validation

Table 4.2: Testing and validation

Test case	Expected result	Actual result	Status
Account creation	100 users created from the CSV	100 accounts present; 100 rows in CSV	Pass
Groups and folders	Four folders, mode 2770, own group	cse(2770), ece(2770), mech(2770), civil(2770)	Pass
Software check	All 8 lab tools found	7 of 8 found; vim missing (reported correctly)	Partial
Cleanup	Only files older than 7 days removed	20 of 30 removed; the 10 newer files kept; 400 KB freed	Pass
Backup creation	`tar.gz` archive created	`backup-20261015-031647.tar.gz`, 707 bytes	Pass
Restore test	Restored file count equals original	4 of 4 files	Pass
Monitoring	CPU, memory, disk and uptime read	1 % CPU, 6 % memory, 3 % disk, load 0.0	Pass
Website build	No type errors; build succeeds	`tsc --noEmit` clean; `vite build` succeeded	Pass

4.2.1 Reproducing the Tests

```bash
</> Bash
sudo ./server/collect.sh
getent passwd | grep -c '^student'
ls -ld /srv/lab/srv/*
ls -lh /srv/lab/backups
npm run typecheck
npm run build
```

4.2.2 Honest Scope of Testing

The administration tests were run once on a single Ubuntu 24.04 VM with one CPU core, so the CPU reading of 1 % reflects an idle machine. The test copied no real student data, which is why the backup is very small. The cleanup test uses files that the script creates itself. The website was checked by type-checking and building it, not by an automated browser test. The LVM commands in section 3.6.4 are a reference procedure and were not run.

4.2.3 Testing Summary

The tests show that the script creates all 100 accounts and four protected department folders, removes only old temporary files, creates a backup that can be restored, reports the health of the server, and that the dashboard builds without errors. The only item that was not fully satisfied is the optional software `vim`, which the check correctly reported as missing.

4.3 Observations

- The script ran from start to finish in one command and produced both a terminal report and `data.json`.

- The setgid bit in mode 2770 keeps new files in the department group, so shared work stays accessible to the whole department.
- The restore test gives real evidence that a backup works, instead of only checking that a file exists.
- The cleanup removed exactly the files older than seven days and kept the newer ones.
- The software check immediately showed that `vim` was not installed.
- Limitations: shared default password, no login on the dashboard, backups on the same disk, and the numbers on the website are those of the last script run, not live.

5. Conclusion and Future Scope

5.1 Conclusion

The College Computer Laboratory Server Management System turns the routine work of a lab administrator into one repeatable script. In a single run it creates 100 student accounts in four department groups, protects each department folder with mode 2770, checks the lab software, removes old temporary files, takes a backup that is proven restorable, and records the health of the server. A React dashboard displays the same results.

The project gave us practical experience in Linux system administration: users and groups, file permissions and the setgid bit, package and process checks, cron automation, backup and recovery, and basic security. We learned that automation must be safe to repeat, that a backup is only useful once its restore has been tested, and that shared default passwords must be replaced in a real deployment.

5.2 Future Scope

- Central login with LDAP or Active Directory instead of local accounts.
- Per-student disk quotas and a quota report on the dashboard.
- Import of the student list from the college database instead of a generated CSV.
- Backups copied to a second server, with automatic removal of old archives.
- E-mail or Slack alerts when disk, memory or CPU usage is high or a restore test fails.
- A small API so the dashboard shows live data instead of the last saved run.
- Automatic installation of missing lab software with `apt`.
- Support for several laboratories managed from one dashboard.

References

1. Black and White Cats and LabServer project source code: https://github.com/allenasat044-prog/labserver

2. Linux manual pages: useradd(8), groupadd(8), chpasswd(8), chage(1), chmod(1), chown(1), find(1), tar(1), stat(1), crontab(5). https://man7.org/linux/man-pages/

3. Ubuntu Server documentation: https://ubuntu.com/server/docs

4. Node.js documentation: https://nodejs.org/docs

5. React, Tailwind CSS and Vite documentation: https://react.dev, https://tailwindcss.com, https://vitejs.dev

6. Course material, Linux System Administration (E1ITA307), Alliance School of Advanced Computing, Alliance University.

Appendices

Appendix A: Dashboard Pages and Data

The website has six pages. It reads `src/data.json` (a copy of the server's `data.json`) and does not change the server.

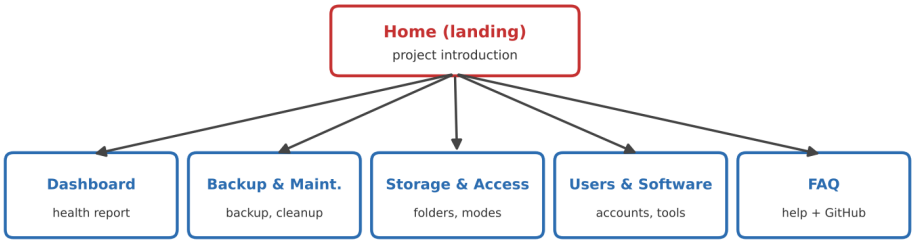


Figure A1: Pages of the web dashboard

Table A.1: Pages and what they show

Page	Shows
Home	Landing page with an introduction to the project
Dashboard	Cards for CPU, memory, disk and logged-in users, a comparison chart, the cleanup donut and the terminal report with a Download .txt button
Backup & Maintenance	Latest backup (file, size, checksum, restore result), automatic cleanup and backup history
Storage & Access	The four department folders with path, mode, group and members
Users & Software	Number of student accounts and the installed software with versions
FAQ	How to use the site, with a link to the GitHub repository

Table A.2: Main fields of data.json

Field	Content
`host`, `os`, `time`, `uptime`	Server name, operating system, run time and uptime
`cpu`, `mem`, `disk`	CPU percentage, cores and load; memory and disk used, total and percentage
`created`, `csv`	Student accounts present and rows in `students.csv`
`soft`	List of tools with version or `null`
`folders`	Department, path, mode, group, file count and users
`cleanup`	Files before, removed, bytes freed, command and time
`backup`	File, size, checksum prefix, files, restored, ok, history
`report`	The full terminal report as text

Appendix B: Troubleshooting

Table B.1: Common problems

Symptom	Fix
`Missing script: "start"`	The project uses `npm run dev`, not `npm start`. Run it inside the folder that contains `package.json`.
`Permission denied` when running `collect.sh`	Make it executable with `chmod +x server/collect.sh`.
`useradd: Permission denied` or no accounts created	Run the script as root: `sudo ./collect.sh`. Without root it does not create users.
Website shows old numbers	Run `sudo npm run refresh` and reload the page.
Port 5173 is busy	Vite chooses the next free port; open the address printed in the terminal.
Landing image or storage video missing	They load from the internet; connect to a network.
`collect.sh` does not work on Windows	It needs Linux. Use WSL (`wsl --install`) or a Linux VM.
Copy data from the VM to the computer	`scp USER@VM_IP:~/labserver/server/data.json ~/labserver/src/data.json`

Appendix C: Scripts

The complete `server/collect.sh` is listed below (long lines are wrapped for printing). The Python version `server/collect.py` in the repository performs the same six steps. The cron entry and the npm `refresh` script are shown in section 3.2.7 and Table 3.5.

</> server/collect.sh

```
#!/usr/bin/env bash
# LabServer collector (bash): does the real work (users, folders, cleanup, backup, restore test)
# prints the health report and writes data.json for the website.
# Run as root on the lab server (cron: 0 2 * * *).
set -u
cd "$(dirname "$0")"

if [ "$(id -u)" -eq 0 ]; then ROOT=true; B=/srv/lab; else ROOT=false; B="$HOME/lab-server"; fi
DEPTS=(cse ece mech civil)
mkdir -p "$B"

j() { printf '%s' "$1" | sed -e 's/\\/\\\\/g' -e 's/"/\\"/g'; }
gb() { awk -v x="$1" 'BEGIN{printf "%.1f GB", x/1073741824}'; }
hs() { awk -v x="$1" 'BEGIN{ if (x>=1048576) printf "%.1f MB", x/1048576; else if (x>=1024) printf "%d KB",
    x/1024; else printf "%d B", x }'; }
now() { date "$1"; }

cpu_pct() {
  read -r _ a1 a2 a3 a4 a5 a6 a7 a8 _ < /proc/stat
  local t1=$((a1+a2+a3+a4+a5+a6+a7+a8)) i1=$((a4+a5))
  sleep 1
  read -r _ b1 b2 b3 b4 b5 b6 b7 b8 _ < /proc/stat
  local t2=$((b1+b2+b3+b4+b5+b6+b7+b8)) i2=$((b4+b5))
  local dt=$((t2-t1)); [ "$dt" -lt 1 ] && dt=1
  awk -v dt="$dt" -v di=$((i2-i1)) 'BEGIN{printf "%d", 100*(1-di/dt)+0.5}'
}
```

```
# 1. Users: 100 students, 25 per department
ROWS=()
for i in $(seq 1 100); do
  ROWS+=("$(printf 'student%03d' "$i"),${DEPTS[$(( (i-1) % 4 ))]}")
done
{ echo "username,department"; printf '%s\n' "${ROWS[@]}"; } > "$B/students.csv"
if $ROOT; then
  for d in "${DEPTS[@]}"; do getent group "lab-$d" >/dev/null || groupadd "lab-$d"; done
  for r in "${ROWS[@]}"; do
    u=${r%,*}; d=${r#*,}
    if ! id "$u" >/dev/null 2>&1; then
      useradd -m -s /bin/bash -g "lab-$d" "$u" && echo "$u:Lab@2026" | chpasswd && chage -d 0 "$u"
    fi
  done
fi
CREATED=$(getent passwd | grep -c '^student')
CSV=${#ROWS[@]}

# 2. Software
TOOLS=(gcc python3 java git node make vim curl)
SOFT_JSON=""; SOFT_OK=0
for t in "${TOOLS[@]}"; do
  if command -v "$t" >/dev/null 2>&1; then
    v=$("$t" --version 2>&1 | head -1 | cut -c1-60); SOFT_OK=$((SOFT_OK+1)); vj="\"$(j "$v")\""
  else
    vj=null
  fi
  SOFT_JSON+="${SOFT_JSON:+,}{\"n\":\"$t\",\"v\":$vj}"
done

# 3. Departmental shared folders
FOLDERS_JSON=""; FOLDERS_TXT=""
for d in "${DEPTS[@]}"; do
  p="$B/srv/$d"; mkdir -p "$p"
  $ROOT && chown ":lab-$d" "$p"
  chmod 2770 "$p"
  grep ",$d\$" "$B/students.csv" | cut -d, -f1 > "$p/members.txt"
  mode=$(stat -c %a "$p"); grp=$(stat -c %G "$p"); nf=$(ls -A "$p" | wc -l)
  FOLDERS_JSON+="${FOLDERS_JSON:+,}{\"dept\":\"$d\",\"path\":\"$p\",\"mode\":\"$mode\",\"group\":\"$grp\",\"
      files\":$nf,\"users\":25}"
  FOLDERS_TXT+="${FOLDERS_TXT:+, }$d($mode)"
done

# 4. Cleanup: seed 30 test files (20 are 10 days old), delete everything older than 7 days
tmp="$B/tmp"; mkdir -p "$tmp"
stamp=$(date +%s)
for i in $(seq 0 29); do
  f="$tmp/lab_${stamp}_$i.tmp"
  head -c 20480 /dev/urandom > "$f"
  if [ "$i" -lt 20 ]; then touch -d "10 days ago" "$f"; else touch -d "1 day ago" "$f"; fi
done
tmp_size() { find "$tmp" -type f -printf '%s\n' | awk '{s+=$1} END{print s+0}'; }
tmp_count() { find "$tmp" -type f | wc -l; }
N0=$(tmp_count); S0=$(tmp_size)
CLEAN_CMD="find $tmp -type f -mtime +7 -delete"
find "$tmp" -type f -mtime +7 -delete
N1=$(tmp_count); S1=$(tmp_size)
REMOVED=$((N0-N1)); FREED=$((S0-S1)); CLEAN_TIME=$(now '+%d %b %Y, %H:%M')

# 5. Backup + restore test
```

```
bd="$B/backups"; mkdir -p "$bd"
ts=$(date +%Y%m%d-%H%M%S); bf="$bd/backup-$ts.tar.gz"
tar -czf "$bf" -C "$B" srv
SRC=$(find "$B/srv" -type f | wc -l)
td=$(mktemp -d)
tar -xzf "$bf" -C "$td"
GOT=$(find "$td/srv" -type f | wc -l)
rm -rf "$td"
SHA=$(sha256sum "$bf" | cut -c1-16)
BSIZE=$(stat -c %s "$bf")
if [ "$SRC" -eq "$GOT" ]; then OK=true; RESULT=PASSED; else OK=false; RESULT=FAILED; fi
HIST_JSON=""
for p in $(ls -1 "$bd"/*.tar.gz | sort | tail -6); do
  HIST_JSON+="${HIST_JSON:+,}{\"file\":\"$(basename "$p")\",\"size\":$(stat -c %s "$p")}"
done
BACKUP_TIME=$(now '+%d %b %Y, %H:%M')

# 6. Monitoring
CPU=$(cpu_pct); CORES=$(nproc)
read -r L1 L2 L3 _ < /proc/loadavg
MT=$(awk '/^MemTotal/{print $2*1024}' /proc/meminfo)
MA=$(awk '/^MemAvailable/{print $2*1024}' /proc/meminfo)
MU=$((MT-MA)); MPCT=$(awk -v u="$MU" -v t="$MT" 'BEGIN{printf "%d", 100*u/t+0.5}')
read -r DT DU < <(df -B1 / | awk 'NR==2{print $2, $3}')
DPCT=$(awk -v u="$DU" -v t="$DT" 'BEGIN{printf "%d", 100*u/t+0.5}')
HOST=$(hostname)
OSNAME=$(. /etc/os-release; echo "$PRETTY_NAME")
UPTIME=$(uptime -p)
TIME=$(now '+%d %b %Y, %H:%M:%S')
WHO_JSON=""
while IFS= read -r line; do [ -n "$line" ] && WHO_JSON+="${WHO_JSON:+,}\"$(j "$line")\""; done < <(who)
ACTIVE_JSON=""
for u in $(ps -eo user= | sort -u); do ACTIVE_JSON+="${ACTIVE_JSON:+,}\"$u\""; done

REPORT="LAB SERVER HEALTH REPORT
Generated : $TIME
Host      : $HOST ($OSNAME)
Uptime    : $UPTIME

CPU       : ${CPU}% of $CORES cores, load [$L1, $L2, $L3]
Memory    : $(gb "$MU") / $(gb "$MT") (${MPCT}%)
Disk (/)  : $(gb "$DU") / $(gb "$DT") (${DPCT}%)

Accounts  : $CREATED student accounts present, $CSV in students.csv
Software  : $SOFT_OK/${#TOOLS[@]} lab tools installed
Folders   : $FOLDERS_TXT
Cleanup   : removed $REMOVED of $N0 temp files, freed $(hs "$FREED")
Backup    : $(basename "$bf") ($(hs "$BSIZE")), restore test $RESULT ($GOT/$SRC files)
"

cat > data.json <<EOF
{
 "host": "$(j "$HOST")",
 "os": "$(j "$OSNAME")",
 "time": "$TIME",
 "uptime": "$(j "$UPTIME")",
 "cpu": {"pct": $CPU, "cores": $CORES, "load": [$L1, $L2, $L3]},
 "mem": {"total": $MT, "used": $MU, "pct": $MPCT},
 "disk": {"total": $DT, "used": $DU, "pct": $DPCT},
 "who": [$WHO_JSON],
 "active": [$ACTIVE_JSON],
```

```
 "root": $ROOT,
 "base": "$B",
 "created": $CREATED,
 "csv": $CSV,
 "soft": [$SOFT_JSON],
 "folders": [$FOLDERS_JSON],
 "cleanup": {"before": $N0, "removed": $REMOVED, "freed": $FREED, "cmd": "$(j "$CLEAN_CMD")", "time":
     "$CLEAN_TIME"},
 "backup": {"file": "$(basename "$bf")", "size": $BSIZE, "sha": "$SHA", "files": $SRC, "restored": $GOT,
     "ok": $OK, "history": [$HIST_JSON], "time": "$BACKUP_TIME"},
 "report": "$(printf '%s' "$REPORT" | sed -e 's/\\/\\\\/g' -e 's/"/\\"/g' | awk '{printf "%s\\n", $0}')"
}
EOF

printf '%s\n' "$REPORT"
```

</> package.json (scripts)

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview",
  "typecheck": "tsc --noEmit",
  "refresh": "cd server && bash collect.sh && cp data.json ../src/data.json"
}
```